

7.1.1 LEAP YEAR

ALGORITHM

Step 1: Start

Step 2: Input integer n

Step 3: If $n \leq 0$

Print "Invalid Input" and Stop

Step 4: Set $i \leftarrow n$

Step 5: While $i \geq 1$

Set $j \leftarrow 1$

While $j \leq i$

Print "*" (without space)

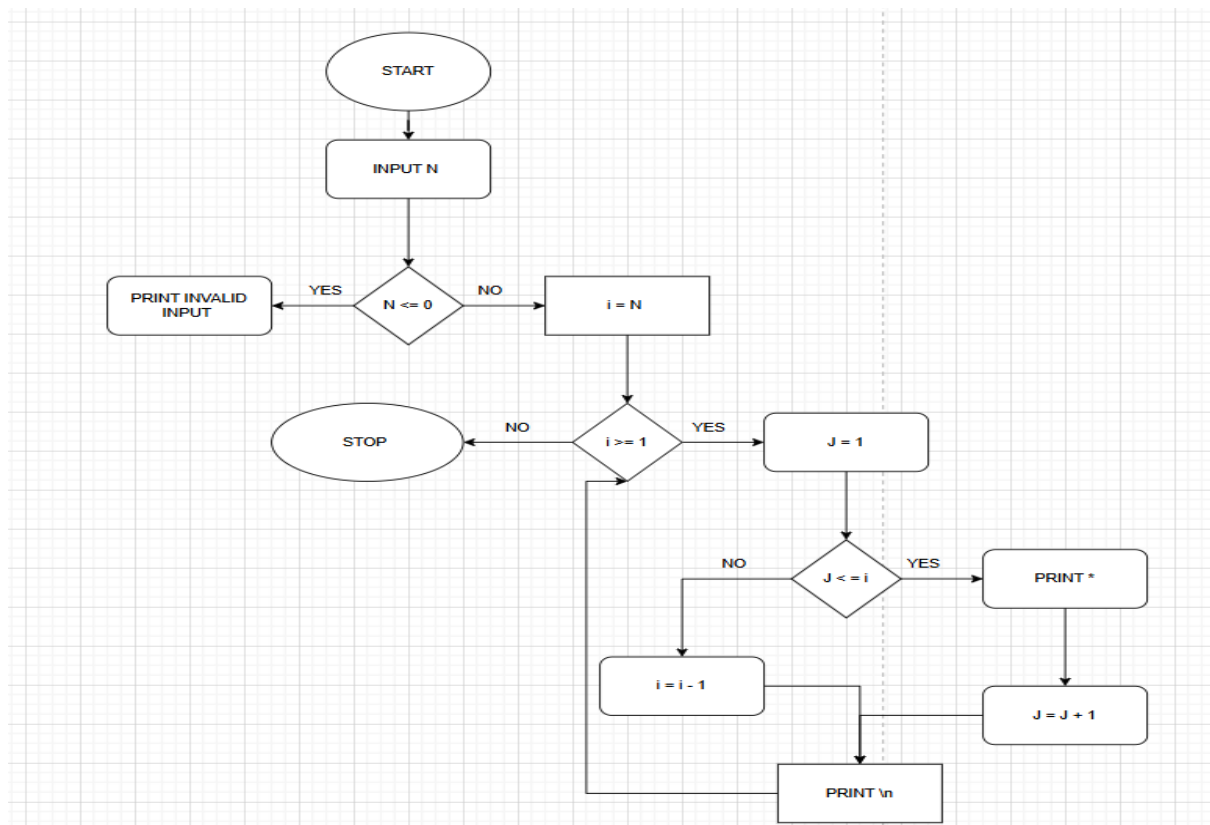
$j \leftarrow j + 1$

Move to next line

$i \leftarrow i - 1$

Step 6: Stop

FLOWCHART



PROGRAM

CODETANTRA

Home

yash.chandak.batch2025@nitnagpur.siu.edu.inSupportLogout

7.1.1. Inverted Star Pattern

Write a Python program to print an inverted right-angled triangle star pattern.

For a given number n , the program should print n rows of stars where the first row contains n stars, the second row contains $n - 1$ stars, and each subsequent row has one less star than the previous row, ending with 1 star in the last row.

All rows should be left-aligned with no spaces between the stars.

Input Format:

- Single line contains an integer n representing the number of rows

Output Format:

- Print an inverted right-angled triangle star pattern with n rows

Note:

- Refer to sample test cases for better understanding.

Sample Test Cases

starPrint.py

1n = int(input())
2for i in range(n, 0, -1):
3 print("*" * i)

Average time0.004 s
4.50 ms

Maximum time0.006 s
6.00 ms

3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1

Expected output

**
*

Actual output

**
*

Test case 2

TerminalTest cases

7.1.2 POSSIBLE COMBINATIONS FROM THREE DIGITS

ALGORITHM

1. Start
2. Input three digits a, b, c
3. Store them in a list `digits = [a, b, c]`
4. For i from 0 to 2
 For j from 0 to 2
 For k from 0 to 2
 If $i \neq j$ AND $j \neq k$ AND $i \neq k$
 Print `digits[i] digits[j] digits[k]`
5. Stop

PROGRAM

7.1.2. Possible Combinations from Three Digits

Write a Python program to accept three digits and print all possible 3-digit combinations that can be formed using those three digits. Each digit should be used exactly once in each combination.

For three digits *a*, *b*, and *c*, there are 6 possible arrangements based on their indices: *abc*, *acb*, *bac*, *bca*, *cab*, *cba*.

Input Format:

- Single line contains three space-separated non-negative integers, *a*, *b*, and *c*, representing the three digits

Output Format:

- Print all possible 3-digit combinations
- Each combination should be printed as a 3-digit number on a separate line
- Print combinations as they appear (maintain leading zeros if first digit is 0)

Constraints:

- $0 \leq \text{each digit} \leq 9$
- Digits can be same or different

Sample Test Cases

```
1 digits = input().split()
2 a, b, c = digits
3 combinations = [
4     a + b + c,
5     a + c + b,
6     b + a + c,
7     b + c + a,
8     c + a + b,
9     c + b + a
10 ]
11
12 for combo in combinations:
13     print(combo)
```

Test Results:

- Average time: 0.008 s (7.86 ms)
- Maximum time: 0.018 s (18.00 ms)
- 3 out of 3 shown test case(s) passed
- 4 out of 4 hidden test case(s) passed

Test case 1	Expected output	Actual output
1 2 3	1 2 3	1 2 3
123	123	123
132	132	132
213	213	213
231	231	231
312	312	312

FLOWCHART

